

Computer Architecture

CS 622 / EE 520

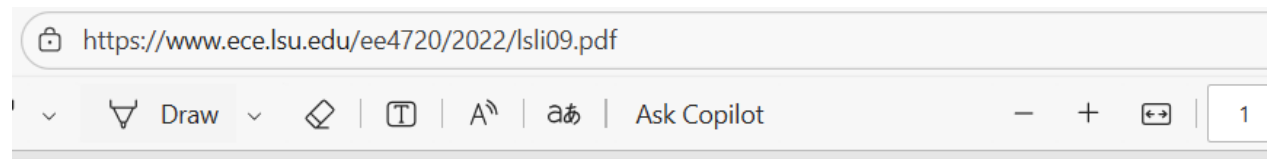
Spring 2026

Lecture 8
Shahid Masud

- Some **Peculiar aspects of RISC-V** Architecture
- **Exception Handling** in Pipeline Architectures
- **Multicycle EX stage**
- **MIPS R4000 Pipeline**
- Solved Examples

Quiz 2
Next Week

Slides Adapted from: (Further Readings)



Reading Notes

Material in This Set

Multicycle Pipeline Operations

Material in This Set

Typical long-latency instructions: mostly floating point

Pipelined v. non-pipelined execution units

FP hardware for the 5-stage MIPS pipeline.

Long-latency implications for hazards, dependencies, and exceptions.

Pipeline diagrams and computation iteration time and CPI.

Readings:
Appendix C of course textbook

More Terminology & Foreshadowing

- **Scalar pipeline:** one insn per stage per cycle
 - Alternative: "superscalar" (later)
- **In-order pipeline:** insns enter execute stage in order
 - Alternative: "out-of-order" (later)
- **Pipeline depth:** number of pipeline stages
 - Nothing magical about five
 - Contemporary high-performance cores have ~15 stage pipelines
 - (even Intel atom, an in-order core, uses 16 stages)

Branches predicted correctly and not Assuming Improved hardware Fig C-12

- BNEZ R1, loop
- Predict Branch not taken and it's not taken

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

- Predict Branch not taken and Woops! it's taken
 - The condition turns out we should take the branch

Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	idle	idle	idle	idle			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

RISC-V Pipeline Peculiar

How Interlock and Forwarding in RISC-V



The process of letting an instruction move from the instruction decode stage (ID) into the execution stage (EX) of this pipeline is usually called instruction issue; an instruction that has made this step is said to have *issued*. For the RISC V integer pipeline, all the data hazards can be checked during the ID phase of the pipeline. If a data hazard exists, the instruction is stalled before it is issued. Likewise, we can determine what forwarding will be needed during ID and set the appropriate controls then. Detecting interlocks early in the pipeline reduces the hardware complexity because the hardware never has to suspend an instruction that has updated the state of the processor, unless the entire processor is stalled. Alternatively, we can detect the hazard or forwarding at the beginning of a clock cycle that uses an operand (EX and MEM for this pipeline). To show the differences in these two



Pipeline Hazard



Situation	Example code sequence	Action
<u>No dependence</u>	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x6, x7	No hazard possible because no dependence exists on x1 in the immediately following three instructions
<u>Dependence requiring stall</u>	ld x1, 45(x2) add x5, x1, x7 sub x8, x6, x7 or x9, x6, x7	Comparators detect the use of x1 in the add and stall the add (and sub and or) before the add begins EX
<u>Dependence overcome by forwarding</u>	ld x1, 45(x2) add x5, x6, x7 sub x8, x1, x7 or x9, x6, x7	Comparators detect use of x1 in sub and forward result of load to ALU in time for sub to begin EX
<u>Dependence with accesses in order</u>	ld x1, 45(x2) add x5, x6, x7 sub x8, x6, x7 or x9, x1, x7	No action required because the read of x1 by or occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half

Figure C.21 Situations that the pipeline hazard detection hardware can see by comparing the destination and sources of adjacent instructions. This table indicates that the only comparison needed is between the destination and the sources on the two instructions following the instruction that wrote the destination. In the case of a stall, the pipeline dependences will look like the third case once execution continues (dependence overcome by forwarding). Of course, hazards that involve x0 can be ignored because the register always contains 0, and the preceding test could be extended to do this.

Eg. Load Interlock

Let's start with implementing the load interlock. If there is a RAW hazard with the source instruction being a load, the load instruction will be in the EX stage when an instruction that needs the load data will be in the ID stage. Thus, we can describe all the possible hazard situations with a small table, which can be directly translated to an implementation. Figure C.22 shows a table that detects all load interlocks when the instruction using the load result is in the ID stage.

Once a hazard has been detected, the control unit must insert the pipeline stall and prevent the instructions in the IF and ID stages from advancing. As we said

Opcode field of ID/EX (ID/EX.IR _{0..5})	Opcode field of IF/ID (IF/ID.IR _{0..6})	Matching operand fields
Load	Register-register ALU, load, store, ALU immediate, or branch	ID/EX.IR[rd] == IF/ID.IR[rs1]
Load	Register-register ALU, or branch	ID/EX.IR[rd] == IF/ID.IR[rs2]

Figure C.22 The logic to detect the need for load interlocks during the ID stage of an instruction requires two comparisons, one for each possible source. Remember that the IF/ID register holds the state of the instruction in ID, which potentially uses the load result, while ID/EX holds the state of the instruction in EX, which is the load instruction.

Exceptions in Pipelines

Why it is really not that this easy? Interrupts, fault, and exception



- The terms interrupt, fault, and exception are used, although not in a consistent fashion. We use the term exception to cover all these mechanisms, including the following:
 - ■ I/ O device request Invoking an operating system service from a user program
 - ■ Tracing instruction execution
 - ■ Breakpoint (programmer-requested interrupt)
 - ■ Integer arithmetic overflow FP arithmetic anomaly
 - ■ Page fault (not in main memory)
 - ■ Misaligned memory accesses (if alignment is required)
 - ■ Memory protection violation
 - ■ Using an undefined or unimplemented instruction
 - ■ Hardware malfunctions
 - ■ Power failure

- Exceptional situations are harder to handle in a pipelined processor because the overlapping of instructions makes it more difficult to know whether an instruction can safely change the state of the processor. In a pipelined processor, an instruction is executed piece by piece and is not completed for several clock cycles. Unfortunately, other instructions in the pipeline can raise exceptions that may force the processor to abort the instructions in the pipeline before they complete. Before we discuss these problems and their solutions in detail, we need to understand what types of situations can arise and what architectural requirements exist for supporting them.
- 1 →
 - 2 →
 - 3 →

Characterization of Exceptions



- **Synchronous vs Asynchronous**

- Event always occurs at the same place with same data and memory
- Asynchronous events are caused by devices external to the processor and memory
- Asynchronous exception can be dealt after completion of current instruction so easier

- **User Requested vs Coerced**

- User requested is predictable and can be dealt after current instruction
- Coerced exceptions are due to hardware events that are not under user program
- Coerced are difficult to implement as they are unpredictable

- **User Maskable vs Non-Maskable**

- The event can be hidden from CPU through Mask register bits

- **Within Instruction or Between Instruction**

- Within instructions are usually synchronous
- Within Instructions are harder to implement as Instructions are stopped and restarted
- Asynchronous Exceptions that occur within instructions always cause program termination

- **Resume vs Terminate**

- Program execution always stops after interrupt or continues after the exception

Five Categories

Actions Required for Different Types of Exceptions

Exception type	Synchronous vs. asynchronous	User request vs. coerced	User maskable vs. nonmaskable	Within vs. between instructions	Resume vs. terminate
I/O device request	Asynchronous	Coerced	Nonmaskable	Between	Resume
Invoke operating system	Synchronous	User request	Nonmaskable	Between	Resume
Tracing instruction execution	Synchronous	User request	User maskable	Between	Resume
Breakpoint	Synchronous	User request	User maskable	Between	Resume
Integer arithmetic overflow	Synchronous	Coerced	User maskable	Within	Resume
Floating-point arithmetic overflow or underflow	Synchronous	Coerced	User maskable	Within	Resume
Page fault	Synchronous	Coerced	Nonmaskable	Within	Resume
Misaligned memory accesses	Synchronous	Coerced	User maskable	Within	Resume
Memory protection violations	Synchronous	Coerced	Nonmaskable	Within	Resume
Using undefined instructions	Synchronous	Coerced	Nonmaskable	Within	Terminate
Hardware malfunctions	Asynchronous	Coerced	Nonmaskable	Within	Terminate
Power failure	Asynchronous	Coerced	Nonmaskable	Within	Terminate

Figure C.26 Five categories are used to define what actions are needed for the different exception types. Exceptions that must allow resumption are marked as resume, although the software may often choose to terminate the program. Synchronous, coerced exceptions occurring within instructions that can be resumed are the most difficult to implement. We might expect that memory protection access violations would always result in termination; however, modern operating systems use memory protection to detect events such as the first attempt to use a page or the first write to a page. Thus, processors should be able to resume after such exceptions.

Pipeline Behaviour Under Exception

from either the target or the fall-through. When an exception occurs, the pipeline control can take the following steps to save the pipeline state safely:

1. Force a trap instruction into the pipeline on the next IF.
2. Until the trap is taken, turn off all writes for the faulting instruction and for all instructions that follow in the pipeline; this can be done by placing zeros into the pipeline latches of all instructions in the pipeline, starting with the instruction that generates the exception, but not those that precede that instruction. This prevents any state changes for instructions that will not be completed before the exception is handled.
3. After the exception-handling routine in the operating system receives control, it immediately saves the PC of the faulting instruction. This value will be used to return from the exception later.

After the exception has been handled, special instructions return the processor from the exception by reloading the PCs and restarting the instruction stream (using the exception return in RISC V). If the pipeline can be stopped so that the instructions just before the faulting instruction are completed and those after it can be restarted from scratch, the pipeline is said to have *precise exceptions*. Ideally, the faulting

Exceptions in RISC-V

Exceptions in RISC V

Figure C.27 shows the RISC V pipeline stages and which problem exceptions might occur in each stage. With pipelining, multiple exceptions may occur in the same clock cycle because there are multiple instructions in execution. For example, consider this instruction sequence:

ld	IF	ID	EX	MEM	WB	
add		IF	ID	EX	MEM	WB

Pipeline stage	Problem exceptions occurring
IF	Page fault on instruction fetch; misaligned memory access; memory protection violation
ID	Undefined or illegal opcode
EX	Arithmetic exception
MEM	Page fault on data fetch; misaligned memory access; memory protection violation
WB	None

Figure C.27 Exceptions that may occur in the RISC V pipeline. Exceptions raised from instruction or data memory access account for six out of eight cases.

Data Page Fault – Multiple Exceptions

ld	IF	ID	EX	MEM	WB	
add		IF	ID	EX	MEM	WB

This pair of instructions can cause a data page fault and an arithmetic exception at the same time, because the `ld` is in the MEM stage while the `add` is in the EX stage. This case can be handled by dealing with only the data page fault and then restarting the execution. The second exception will reoccur (but not the first, if the software is correct), and when the second exception occurs it can be handled independently.

Exception that is Out of Order

ld	IF	ID	EX	MEM	WB	
add		IF	ID	EX	MEM	WB

In reality, the situation is not as straightforward as this simple example. Exceptions may occur out of order; that is, **an instruction may cause an exception before an earlier instruction causes one**. Consider again the preceding sequence of instructions, ld followed by add. **The ld can get a data page fault, seen when the instruction is in MEM, and the add can get an instruction page fault, seen when the add instruction is in IF. The instruction page fault will actually occur first, even though it is caused by a later instruction!**

Because we are implementing precise exceptions, the pipeline is required to handle the exception caused by the `ld` instruction first. To explain how this works, let's call the instruction in the position of the `ld` instruction i , and the instruction in the position of the `add` instruction $i+1$. The pipeline cannot simply handle an exception when it occurs in time, because that will lead to exceptions occurring out of the unpipelined order. Instead, the hardware posts all exceptions caused by a given instruction in a status vector associated with that instruction. The exception status vector is carried along as the instruction goes down the pipeline. Once an exception indication is set in the exception status vector, any control signal that may cause a data value to be written is turned off (this includes both register writes and memory writes). Because a store can cause an exception during MEM, the hardware must be prepared to prevent the store from completing if it raises an exception.

When an instruction enters WB (or is about to leave MEM), the exception status vector is checked. If any exceptions are posted, they are handled in the order in which they would occur in time on an unpipelined processor—the exception corresponding to the earliest instruction (and usually the earliest pipe stage for that instruction) is handled first. This guarantees that all exceptions will be seen on instruction i before any are seen on $i + 1$. Of course, any action taken in earlier pipe stages on behalf of instruction i may be invalid, but because writes to the register file and memory were disabled, no state could have been changed. As we will see in [Section C.5](#), maintaining this precise model for FP operations is much harder.

Instruction Set Complications

→ No RISC V instruction has more than one result, and our RISC V pipeline writes that result only at the end of an instruction's execution. When an instruction is guaranteed to complete, it is called *committed*. In the RISC V integer pipeline, all instructions are committed when they reach the end of the MEM stage (or beginning of WB) and no instruction updates the state before that stage. Thus, precise exceptions are straightforward. Some processors have instructions that change the state in the middle of the instruction execution, before the instruction and its predecessors are guaranteed to complete. For example, autoincrement addressing modes in the IA-32 architecture cause the update of registers in the middle of an instruction execution. In such a case, if the instruction is aborted because of an exception, it will leave the processor state altered. Although we know which instruction caused the exception, without additional hardware support the exception will be imprecise because the instruction will be half finished. Restarting the instruction stream after such an imprecise exception is difficult. Alternatively, we could avoid updating the state

Handling Complex EX Stage with Multiple cycles

Complex Floating Point Stage in ALU

End of Innocence – Floating Point in 1 Cycle

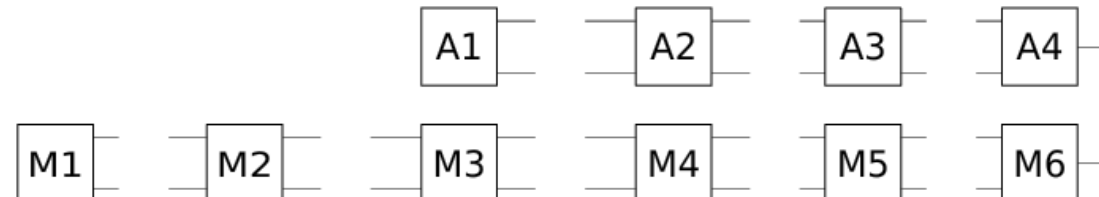
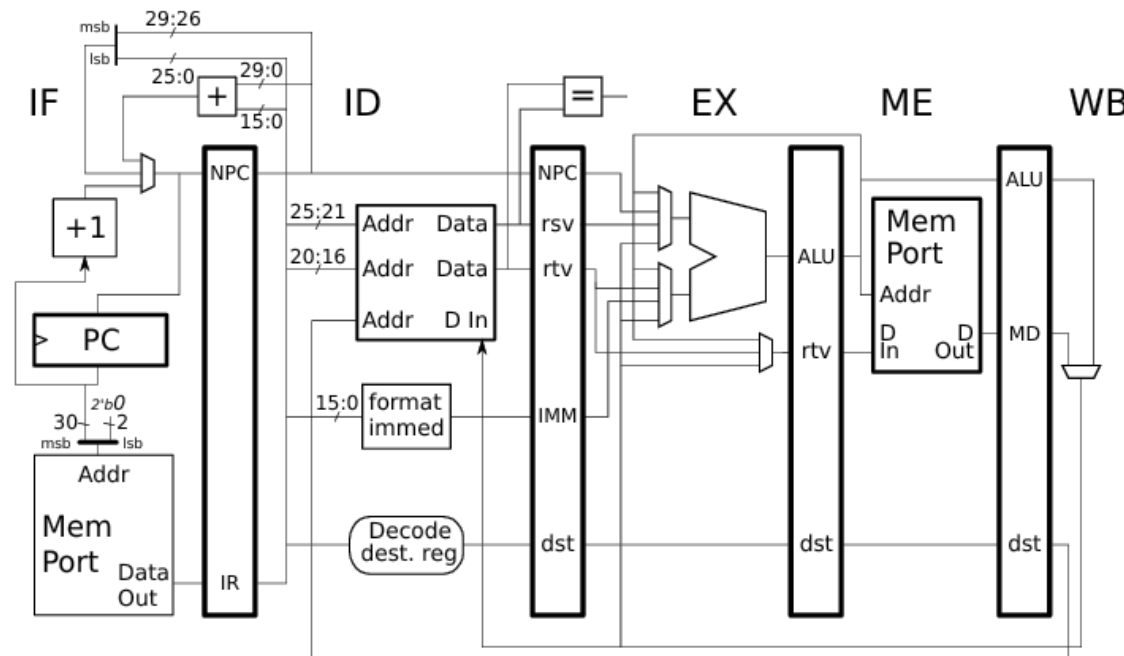
The End of Innocence

Unfortunately we must now set aside this simplicity and elegance.

Because floating-point operations ...

... can't feasibly be computed in one or two cycles.

The challenge is putting these units in our pipeline.



Pipeline Architecture with Multiple Execution

EX here is Not Pipelined

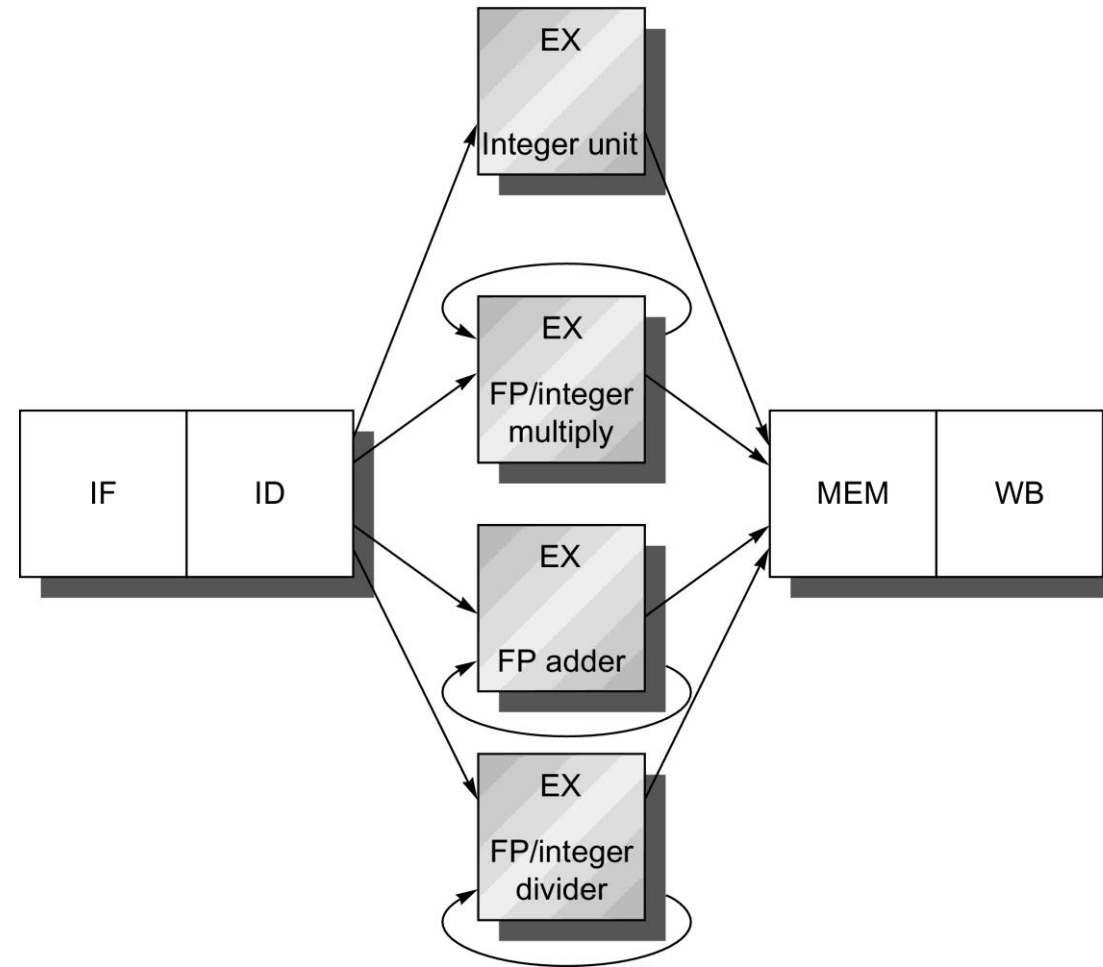


Figure C.28 The RISC V pipeline with three additional unpipelined, floating-point, functional units.

Because only one instruction issues on every clock cycle, all instructions go through the standard pipeline for integer operations. The FP operations simply loop when they reach the EX stage. After they have finished the EX stage, they proceed to MEM and WB to complete execution.

Multicycle EX stage – FP Example



It is impractical to require that all RISC V FP operations complete in 1 clock cycle, or even in 2. Doing so would mean accepting a slow clock or using enormous amounts of logic in the FP units, or both. Instead, the FP pipeline will allow for a longer latency for operations. This is easier to grasp if we imagine the FP instructions as having the same pipeline as the integer instructions, with two important changes. First, the EX cycle may be repeated as many times as needed to complete the operation—the number of repetitions can vary for different operations. Second, there may be multiple FP functional units. A stall will occur if the instruction to be issued will cause either a structural hazard for the functional unit it uses or a data hazard.

Assume Four Functional Units in EX



For this section, let's assume that there are four separate functional units in our RISC V implementation:

1. The main integer unit that handles loads and stores, integer ALU operations, and branches
2. FP and integer multiplier
3. FP adder that handles FP add, subtract, and conversion
4. FP and integer divider

Latency of Different EX Units

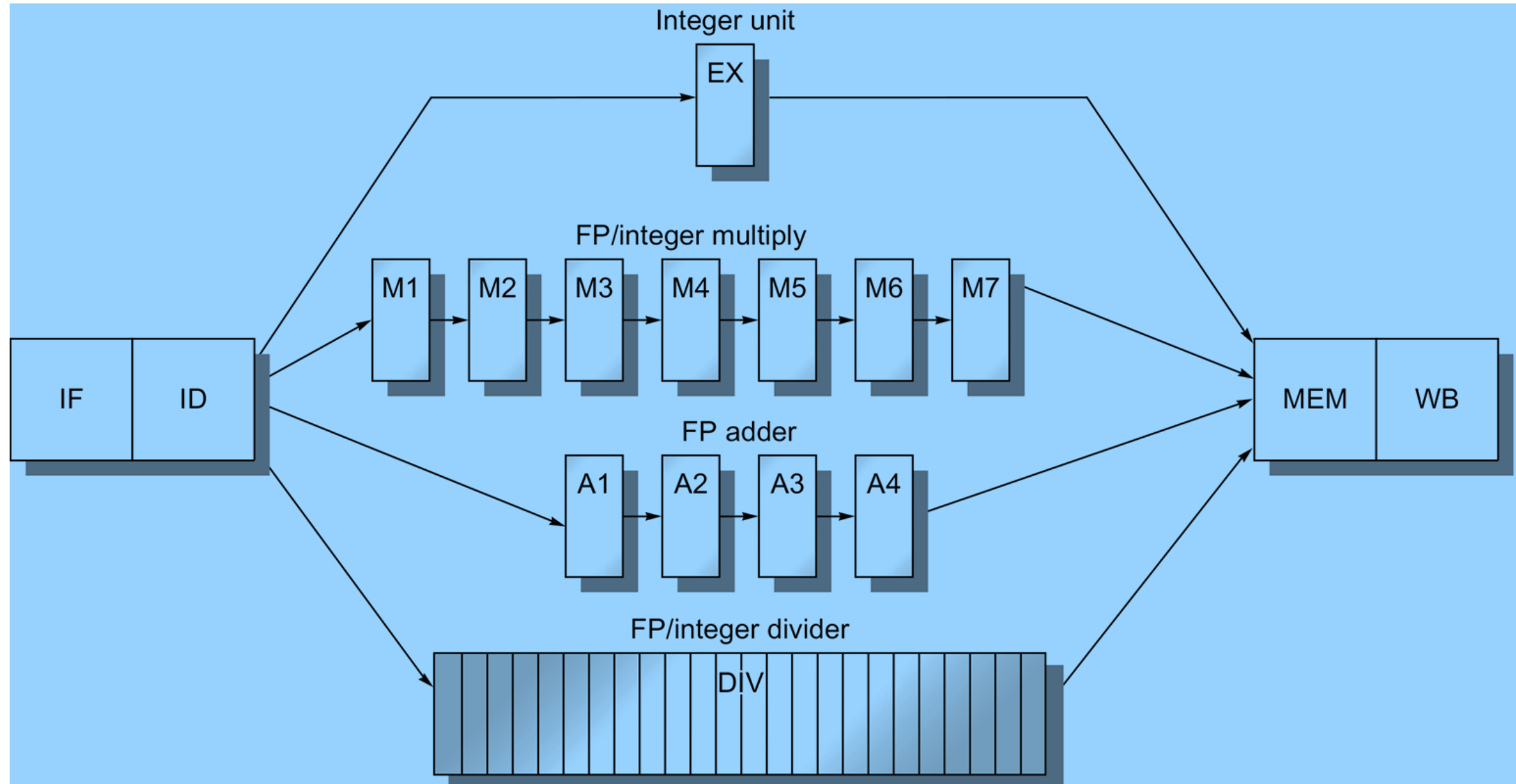
In reality, the intermediate results are probably not cycled around the EX unit as Figure C.28 suggests; instead, the EX pipeline stage has some number of clock delays larger than 1. We can generalize the structure of the FP pipeline shown in Figure C.28 to allow pipelining of some stages and multiple ongoing operations.

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Figure C.29 Latencies and initiation intervals for functional units.

With this definition of latency, integer ALU operations have a latency of 0, because the results can be used on the next clock cycle, and loads have a latency of 1, because their results can be used after one intervening cycle. Because most operations consume their operands at the beginning of EX, the latency is usually the number of stages after EX that an instruction produces a result—for example, zero stages for ALU operations and one stage for loads. The primary exception is stores, which consume the value being stored one cycle later. Hence, the latency to a store for the value being stored, but not for the base address register, will be one cycle less. Pipeline latency is essentially equal to one cycle less than the depth of the execution pipeline, which is the number of stages from the EX stage to the stage that produces the result. Thus, for the preceding example pipeline, the number of stages in an FP add is four, while the number of stages in an FP multiply is seven. To achieve a higher clock rate, designers need to put fewer logic levels in each pipe stage, which makes the number of pipe stages required for more complex operations larger. The penalty for the faster clock rate is thus longer latency for operations.

Include Multiple Operations in Pipeline



Multiple Outstanding FP Operations in Pipeline

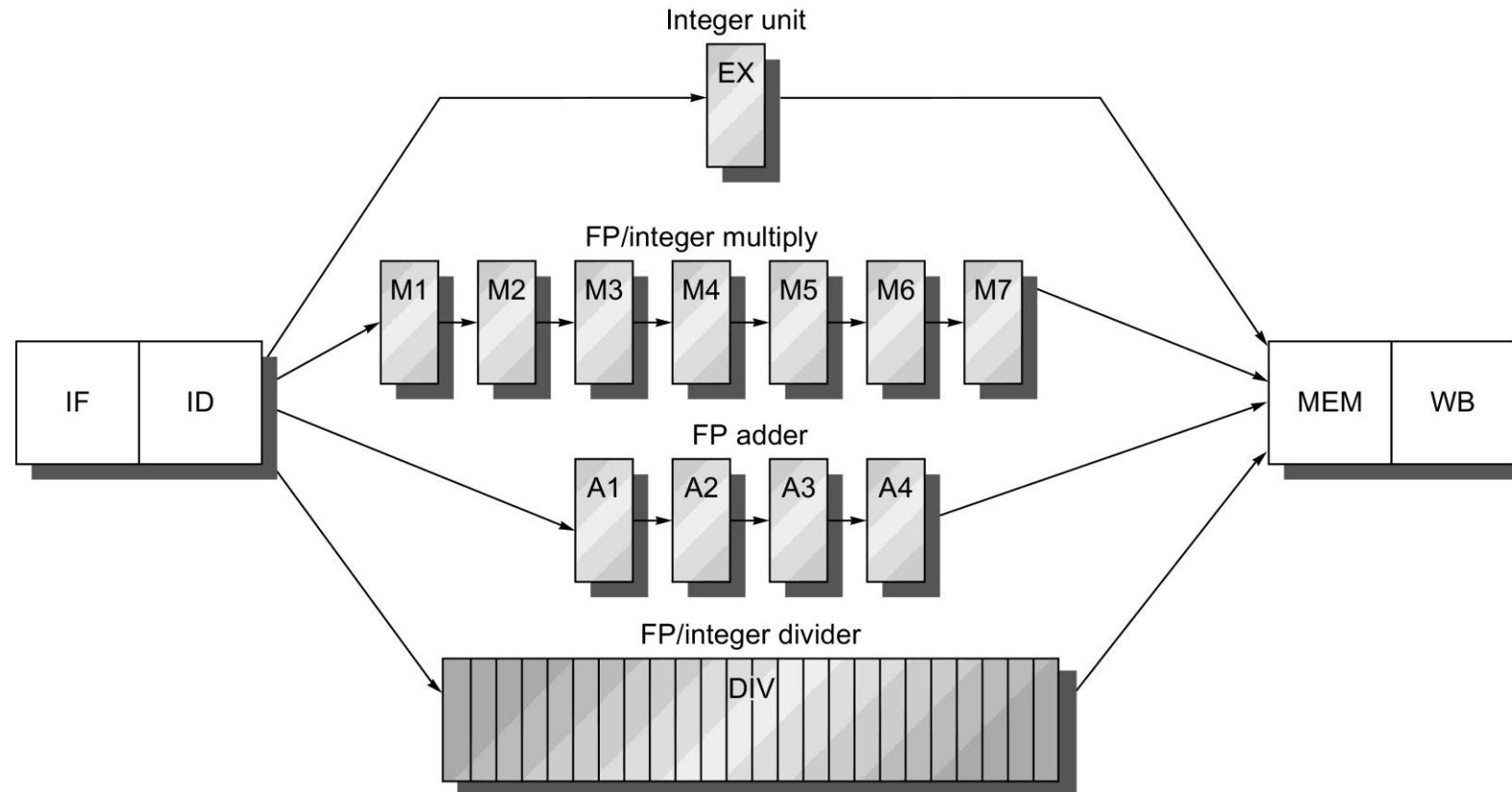
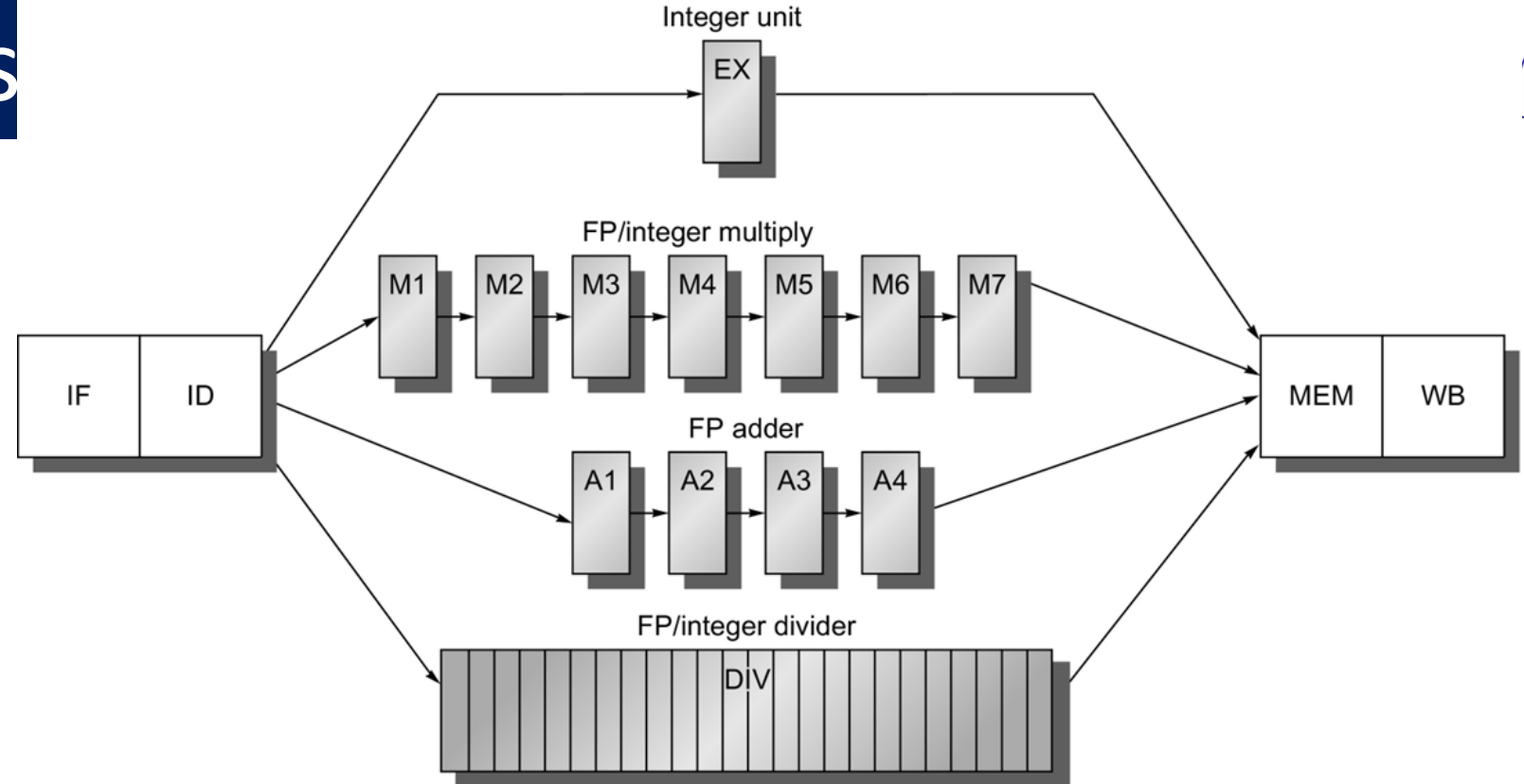


Figure C.30 A pipeline that supports multiple outstanding FP operations. The FP multiplier and adder are fully pipelined and have a depth of seven and four stages, respectively. The FP divider is not pipelined, but requires 24 clock cycles to complete. The latency in instructions between the issue of an FP operation and the use of the result of that operation without incurring a RAW stall is determined by the number of cycles spent in the execution stages. For example, the fourth instruction after an FP add can use the result of the FP add. For integer ALU operations, the depth of the execution pipeline is always one and the next instruction can use the results.

Adding Registers



The structure of the pipeline in Figure C.30 requires the introduction of the additional pipeline registers (e.g., A1/A2, A2/A3, A3/A4) and the modification of the connections to those registers. The ID/EX register must be expanded to connect ID to EX, DIV, M1, and A1; we can refer to the portion of the register associated with one of the next stages with the notation ID/EX, ID/DIV, ID/M1, or ID/A1. The pipeline register between ID and all the other stages may be thought of as logically separate registers and may, in fact, be implemented as separate registers. Because only one operation can be in a pipe stage at a time, the control information can be associated with the register at the head of the stage.

Complete Architecture incl MC FP instruction

Highest Cost: *Fully Pipelined Functional Unit*

In a fully pipelined unit all stages have an initiation interval of 1.

Example, fully pipelined FP add:

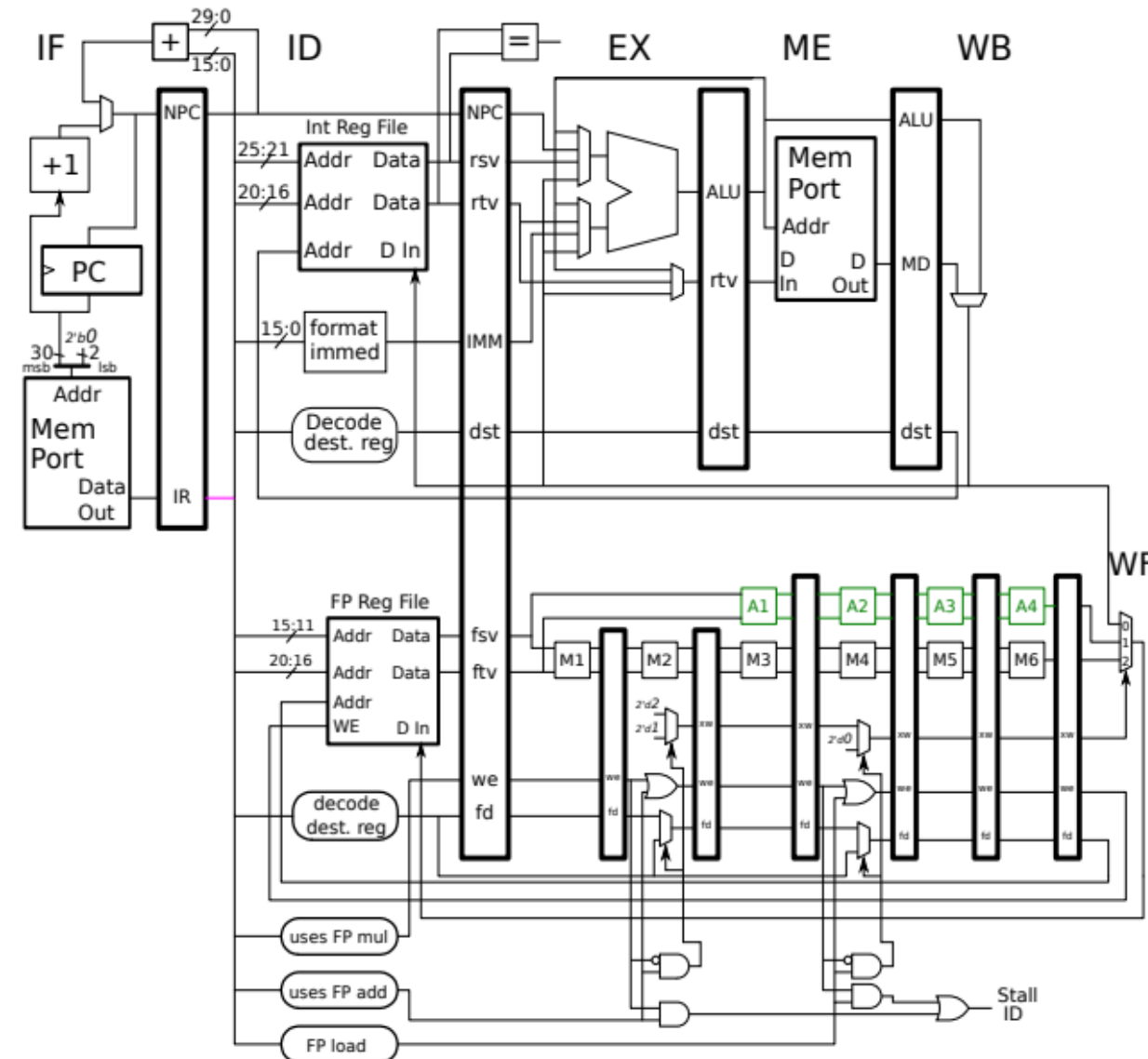
# Cycle	0	1	2	3	4	5	6	7	8	9	10	11	12
add.d f0, f2, f4	IF	ID	A1	A2	A3	A4	WF						
add.d f6, f8, f10		IF	ID	A1	A2	A3	A4	WF					
add.d f12, f14, f16			IF	ID	A1	A2	A3	A4	WF				
add.d f18, f12, f20				IF	ID	----->	A1	A2	A3	A4	WF		

The add.d f18 stalls due to a dependence.

The initiation interval is 1. (True for all fully pipelined units.)

The add operation latency is 4. (True for this FP adder unit.)

Typically addition and multiplication units are fully pipelined.



Breaking Our Assumption of Integer Pipeline

- The divide unit is not fully pipelined
 - structural hazards can occur
 - need to be detected and stall incurred.
- The instructions have varying running times
 - the number of register writes required in a cycle can be > 1
- Instructions no longer reach WB in order
 - Write after write (WAW) hazards are possible
 - Note that write after read (WAR) hazards are not possible, since the register reads always occur in ID.
- Instructions can complete in a different order than they were issued (out-of-order complete)
 - causing problems with exceptions
- Longer latency of operations
 - stalls for RAW hazards will be more frequent.

Timing of Independent Pipelined FP Operations



fmul.d	IF	ID	<i>M1</i>	M2	M3	M4	M5	M6	<i>M7</i>	MEM	WB
fadd.d		IF	ID	<i>A1</i>	A2	A3	<i>A4</i>	<i>MEM</i>	WB		
?? fadd.d			IF	ID	<i>EX</i>	<i>MEM</i>	WB				
fsd				IF	ID	<i>EX</i>	<i>MEM</i>	WB			

Figure C.31 The pipeline timing of a set of independent FP operations. The stages in *italics* show where data are needed, while the stages in **bold** show where a result is available. FP loads and stores use a 64-bit path to memory so that the pipelining timing is just like an integer load or store.

Hazard and Forwarding in Longer Latency EX



There are a number of different aspects to the hazard detection and forwarding for a pipeline like that shown in Figure C.30.

1. Because the divide unit is not fully pipelined, **structural hazards** can occur. These will need to be detected and issuing instructions will need to be stalled.
2. Because the **instructions have varying running times**, the **number of register writes required in a cycle can be larger than 1**.
3. **Write after write (WAW) hazards** are possible, because **instructions no longer reach WB in order**. Note that **write after read (WAR) hazards are not possible**, because the **register reads always occur in ID**.
4. **Instructions can complete in a different order than they were issued**, causing **problems with exceptions**; we deal with this in the next subsection.
5. Because of **longer latency of operations**, **stalls for RAW hazards will be more frequent**.

Stalls due to RAW Hazards in FP stage

The increase in stalls arising from longer operation latencies is fundamentally the same as that for the integer pipeline. Before describing the new problems that arise in this FP pipeline and looking at solutions, let's examine the **potential impact of RAW hazards**. Figure C.32 shows a typical FP code sequence and the resultant stalls. At the end of this section, we'll examine the performance of this FP pipeline for our SPEC subset.

		Clock cycle number																
Instruction		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Stored dbl prec	f1d f4,0(x2)	IF	ID	EX	MEM	WB												
	fmul.d f0,f4,f6		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
	fadd.d f2,f0,f8			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
	fsd f2,0(x2)					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

Figure C.32 A typical FP code sequence showing the stalls arising from RAW hazards. The **longer pipeline substantially raises the frequency of stalls versus the shallower integer pipeline**. Each instruction in this sequence is dependent on the previous and proceeds as soon as data are available, which assumes the pipeline has full bypassing and forwarding. The `fsd` must be stalled an extra cycle so that its MEM does not conflict with the `fadd.d`. Extra hardware could easily handle this case.

Now look at the problems arising from writes, described as (2) and (3) in the earlier list. If we assume that the FP register file has one write port, sequences of FP operations, as well as an FP load together with FP operations, can cause conflicts for the register write port. Consider the pipeline sequence shown in Figure C.33. In clock cycle 11, all three instructions will reach WB and want to write the register file. With only a single register file write port, the processor must serialize the instruction completion. This single register port represents a structural hazard. We could increase the number of write ports to solve this, but that solution may be unattractive because the additional write ports would be used only rarely. This is because the maximum steady-state number of write ports needed is 1. Instead, we choose to detect and enforce access to the write port as a structural hazard.

Implement this Interlock

There are two different ways to implement this interlock. The first is to track the use of the write port in the ID stage and to stall an instruction before it issues, just as we would for any other structural hazard. Tracking the use of the write port can be done with a shift register that indicates when already-issued instructions will use the register file. If the instruction in ID needs to use the register file at the same time as an instruction already issued, the instruction in ID is stalled for a cycle. On each clock the reservation register is shifted 1 bit. This implementation has an advantage: It maintains the property that all interlock detection and stall insertion occurs in the ID stage. The cost is the addition of the shift register and write conflict logic. We will assume this scheme throughout this section.

An alternative scheme is to stall a conflicting instruction when it tries to enter either the MEM or WB stage. If we wait to stall the conflicting instructions until they want to enter the MEM or WB stage, we can choose to stall either instruction. A simple, though sometimes suboptimal, heuristic is to give priority to the unit with the longest latency, because that is the one most likely to have caused another instruction to be stalled for a RAW hazard. The advantage of this scheme is that it does not require us to detect the conflict until the entrance of the MEM or WB stage, where it is easy to see. The disadvantage is that it complicates pipeline control, as stalls can now arise from two places. Notice that stalling before entering MEM will cause the EX, A4, or M7 stage to be occupied, possibly forcing the stall to trickle back in the pipeline. Likewise, stalling before WB would cause MEM to back up.

Our other problem is the possibility of WAW hazards. To see that these exist, consider the example in [Figure C.33](#). If the `fadd.d` instruction were issued one cycle earlier and had a destination of `f2`, then it would create a WAW hazard, because it would write `f2` one cycle earlier than the `fadd.d`. Note that this hazard only occurs when the result of the `fadd.d` is overwritten *without* any instruction ever using it! If there were a use of `f2` between the `fadd.d` and the `fadd.d`, the pipeline would need to be stalled for a RAW hazard, and the `fadd.d` would not issue until the `fadd.d` was completed. We could argue that, for our pipeline, WAW hazards only occur when a useless instruction is executed, but we must still detect them and make sure that the result of the `fadd.d` appears in `f2` when we are done. (As we will see in [Section C.8](#), such sequences sometimes *do* occur in reasonable code.)

Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
<code>fmul.d f0, f4, f6</code>	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM
...		IF	ID	EX	MEM	WB				
...			IF	ID	EX	MEM	WB			
<code>fadd.d f2, f4, f6</code>				IF	ID	A1	A2	A3	A4	MEM
...					IF	ID	EX	MEM	WB	
...						IF	ID	EX	MEM	WB
<code>fld f2, 0(x2)</code>							IF	ID	EX	MEM

Figure C.33 Three instructions want to perform a write-back to the FP register file simultaneously, as shown in **clock cycle 11**. This is *not* the worst case, because an earlier divide in the FP unit could also finish on the same clock. Note that although the `fmul.d`, `fadd.d`, and `fld` are in the MEM stage in clock cycle 10, only the `fld` actually uses the memory, so no structural hazard exists for MEM.

How to handle this WAW Hazard?



There are two possible ways to handle this WAW hazard. The first approach is to delay the issue of the load instruction until the `fadd.d` enters MEM. The second approach is to stamp out the result of the `fadd.d` by detecting the hazard and changing the control so that the `fadd.d` does not write its result. Then the `fadd.d` can issue right away. Because this hazard is rare, either scheme will work fine—you can pick whatever is simpler to implement. In either case, the hazard can be detected during ID when the `fadd.d` is issuing, and stalling the `fadd.d` or making the `fadd.d` a no-op is easy. The difficult situation is to detect that the `fadd.d` might finish before the `fadd.d`, because that requires knowing the length of the pipeline and the current position of the `fadd.d`. Luckily, this code sequence (two writes with no intervening read) will be very rare, so we can use a simple solution: if an instruction in ID wants to write the same register as an instruction already issued, do not issue the instruction to EX. In

What To check while issuing an Instruction?



sional moves needed between the two register sets.) Assuming that the pipeline does all hazard detection in ID, there are **three checks that must be performed before an instruction can issue:**

1. **Check for structural hazards**—Wait until the required functional unit is not busy (this is only needed for divides in this pipeline) and make sure the register write port is available when it will be needed.
2. **Check for a RAW data hazard**—Wait until the source registers are not listed as pending destinations in a pipeline register that will not be available when this instruction needs the result. A number of checks must be made here, depending on both the source instruction, which determines when the result will be available, and the destination instruction, which determines when the value is needed. For example, if the instruction in ID is an FP operation with source register f2, then f2 cannot be listed as a destination in ID/A1, A1/A2, or A2/A3, which correspond to FP add instructions that will not be finished when the instruction in ID needs a result. (ID/A1 is the portion of the output register of ID that is sent to A1.) Divide is somewhat more tricky, if we want to allow the last few cycles of a divide to be overlapped, because we need to handle the case when a divide is close to finishing as special. In practice, designers might ignore this optimization in favor of a simpler issue test.
3. **Check for a WAW data hazard**—Determine if any instruction in A1, ..., A4, D, M1, ..., M7 has the same register destination as this instruction. If so, stall the issue of the instruction in ID.

Out of Order Completion



Another problem caused by these long-running instructions can be illustrated with the following sequence of code:

```
fdiv.d      f0, f2, f4
fadd.d      f10, f10, f8
fsub.d      f12, f12, f14
```

This code sequence looks straightforward; there are no dependences. A problem arises, however, because an instruction issued early may complete after an instruction issued later. In this example, we can expect `fadd.d` and `fsub.d` to complete before the `fdiv.d` completes. This is called out-of-order completion and is common in pipelines with long-running operations (see [Section C.7](#)). Because hazard



Problem with Out-of-Order Completion



```
fdiv.d      f0,f2,f4
fadd.d      f10,f10,f8
fsub.d      f12,f12,f14
```

why is out-of-order completion a problem? Suppose that the `fsub.d` causes a floating-point arithmetic exception at a point where the `fadd.d` has completed but the `fdiv.d` has not. The result will be an imprecise exception, something we are trying to avoid. It may appear that this could be handled by letting the floating-point pipeline drain, as we do for the integer pipeline. But the exception may be in a position where this is not possible. For example, if the `fdiv.d` decided to take a floating-point-arithmetic exception after the add completed, we could not have a precise exception at the hardware level. In fact, because the `fadd.d` destroys one of its operands, we could not restore the state to what it was before the `fdiv.d`, even with software help.



Solution to out of order completion



1 than they were issued. There are four possible approaches to dealing with out-of-order completion. The first is to ignore the problem and settle for imprecise exceptions. This approach was used in the 1960s and early 1970s. It was still used in some supercomputers in the past fifteen years, where certain classes of exceptions were not allowed or were handled by the hardware without stopping the pipeline. It is difficult to use this approach in most processors built today because of

2 A second approach is to buffer the results of an operation until all the operations that were issued earlier are complete. Some processors actually use this solution, but it becomes expensive when the difference in running times among operations is large, because the number of results to buffer can become large. Furthermore,

3 A third technique in use is to allow the exceptions to become somewhat imprecise, but to keep enough information so that the trap-handling routines can create a precise sequence for the exception. This means knowing what operations were in the pipeline and their PCs. Then, after handling the exception, the software finishes any instructions that precede the latest instruction completed, and the sequence can restart. Consider the following worst-case code sequence:

4 The final technique is a hybrid scheme that allows the instruction issue to continue only if it is certain that all the instructions before the issuing instruction will complete without causing an exception. This guarantees that when an exception occurs, no instructions after the interrupting one will be completed and all of the instructions before the interrupting one can be completed. This sometimes

MIPS R4000 Deep Pipeline Case

8 Stage Pipeline of MIPS R4000

- IF: First half of instruction fetch; PC selection actually happens here, together with initiation of instruction cache access.
- IS: Second half of instruction fetch, complete instruction cache access.
- RF: Instruction decode and register fetch, hazard checking, and instruction cache hit detection.
- EX: Execution, which includes effective address calculation, ALU operation, and branch-target computation and condition evaluation.
- DF: Data fetch, first half of data cache access.
- DS: Second half of data fetch, completion of data cache access.
- TC: Tag check, to determine whether the data cache access hit.
- WB: Write-back for loads and register-register operations.

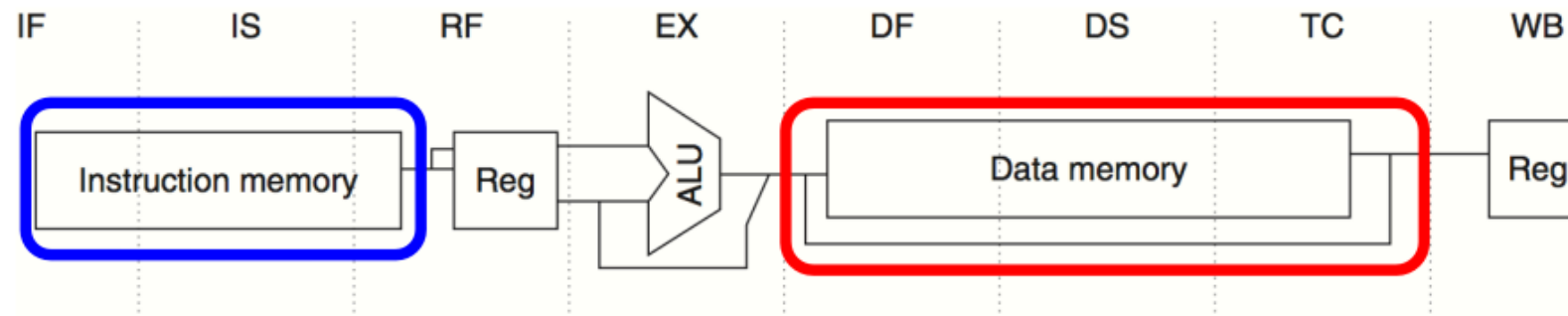
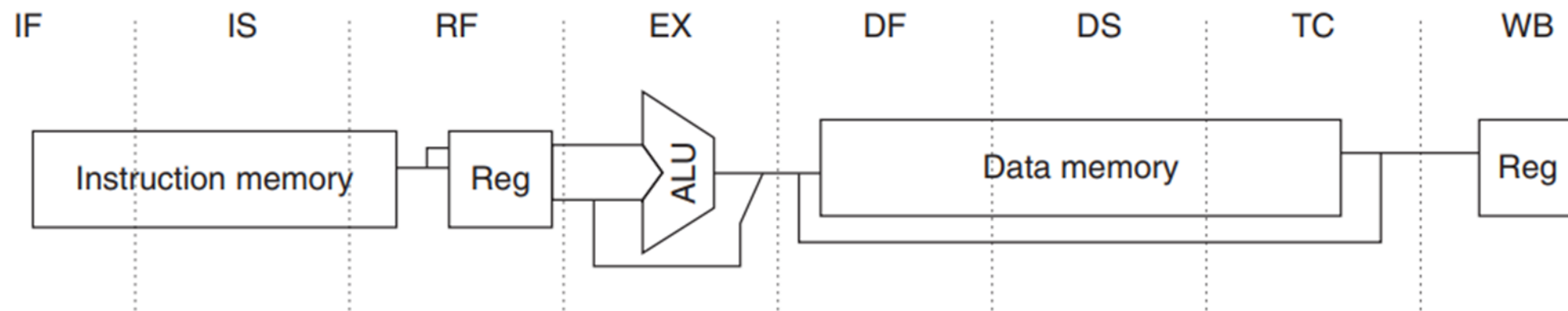


Figure C.36 The eight-stage pipeline structure of the R4000 uses pipelined instruction and data caches. The pipe stages are labeled and their detailed function is described in the text. The vertical dashed lines represent the stage boundaries as well as the location of pipeline latches. The instruction is actually available at the end of IS, but the tag check is done in RF, while the registers are fetched. Thus, we show the instruction memory as operating through RF. The TC stage is needed for data memory access, because we cannot write the data into the register until we know whether the cache access was a hit or not.

Complexity in Architecture

In addition to substantially increasing the amount of forwarding required, this longer-latency pipeline increases both the load and branch delays. Figure C.37 shows that load delays are two cycles, because the data value is available at the end of DS. Figure C.38 shows the shorthand pipeline schedule when a use immediately follows a load. It shows that forwarding is required for the result of a load instruction to a destination that is three or four cycles later.



2 Cycle Load Delay

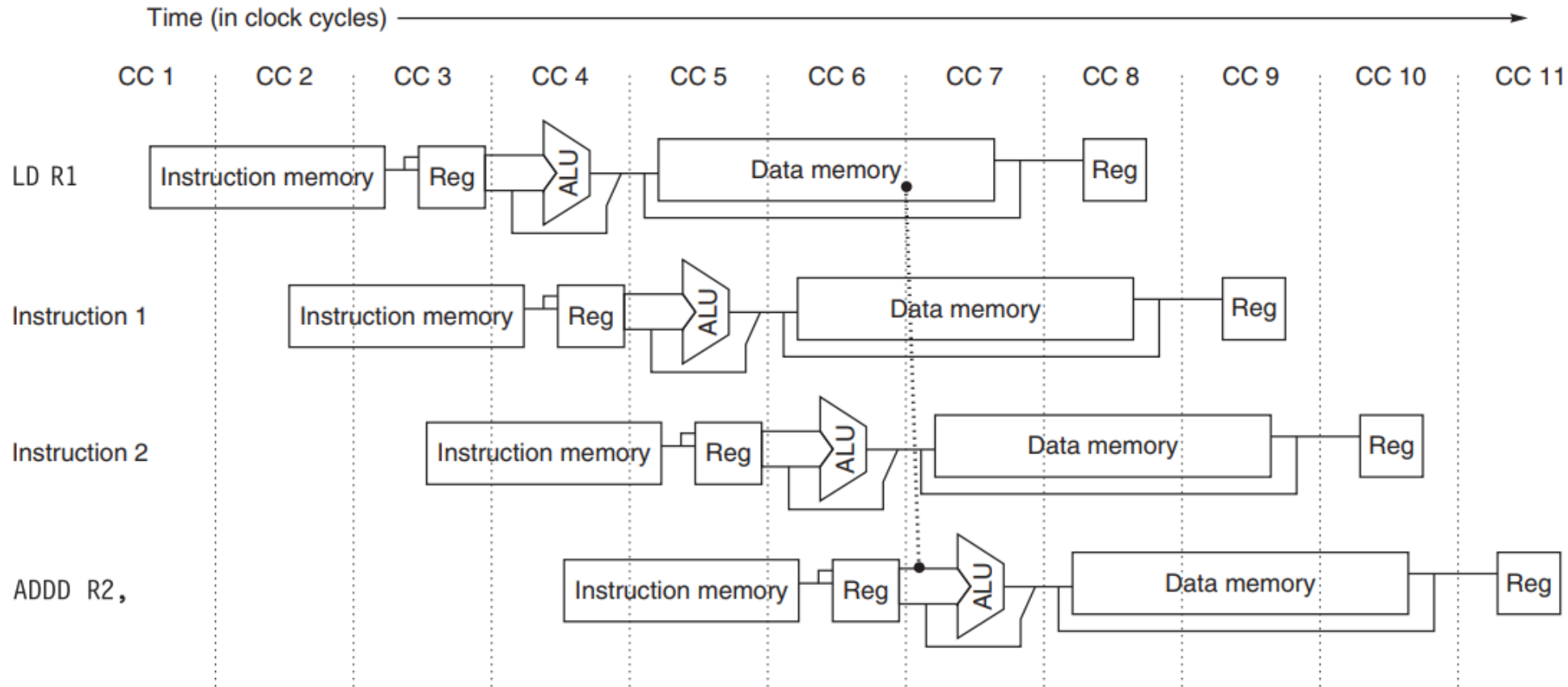


Figure C.37 The structure of the R4000 integer pipeline leads to a x1 load delay. A x1 delay is possible because the data value is available at the end of DS and can be bypassed. If the tag check in TC indicates a miss, the pipeline is backed up a cycle, when the correct data are available.

Forwarding Example MIPS 4000

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
ld x1,...	IF	IS	RF	EX	DF	DS	TC	WB	
add x2,x1,...		IF	IS	RF	Stall	Stall	EX	DF	DS
sub x3,x1,...			IF	IS	Stall	Stall	RF	EX	DF
or x4,x1,...				IF	Stall	Stall	IS	RF	EX

Figure C.38 A load instruction followed by an immediate use results in a x1 stall. Normal forwarding paths can be used after two cycles, so the `add` and `sub` get the value by forwarding after the stall. The `or` instruction gets the value from the register file. Because the two instructions after the load could be independent and hence not stall, the bypass can be to instructions that are three or four cycles after the load.

Figure C.39 shows that the basic branch delay is three cycles, because the branch condition is computed during EX. The MIPS architecture has a single-cycle delayed branch. The R4000 uses a predicted-not-taken strategy for the remaining two cycles of the branch delay. As Figure C.40 shows, untaken branches are simply one-cycle delayed branches, while taken branches have a one-cycle delay slot followed by two idle cycles. The instruction set provides a branch-likely instruction, which we described earlier and which helps in filling the branch delay slot.

Basic Branch Delay

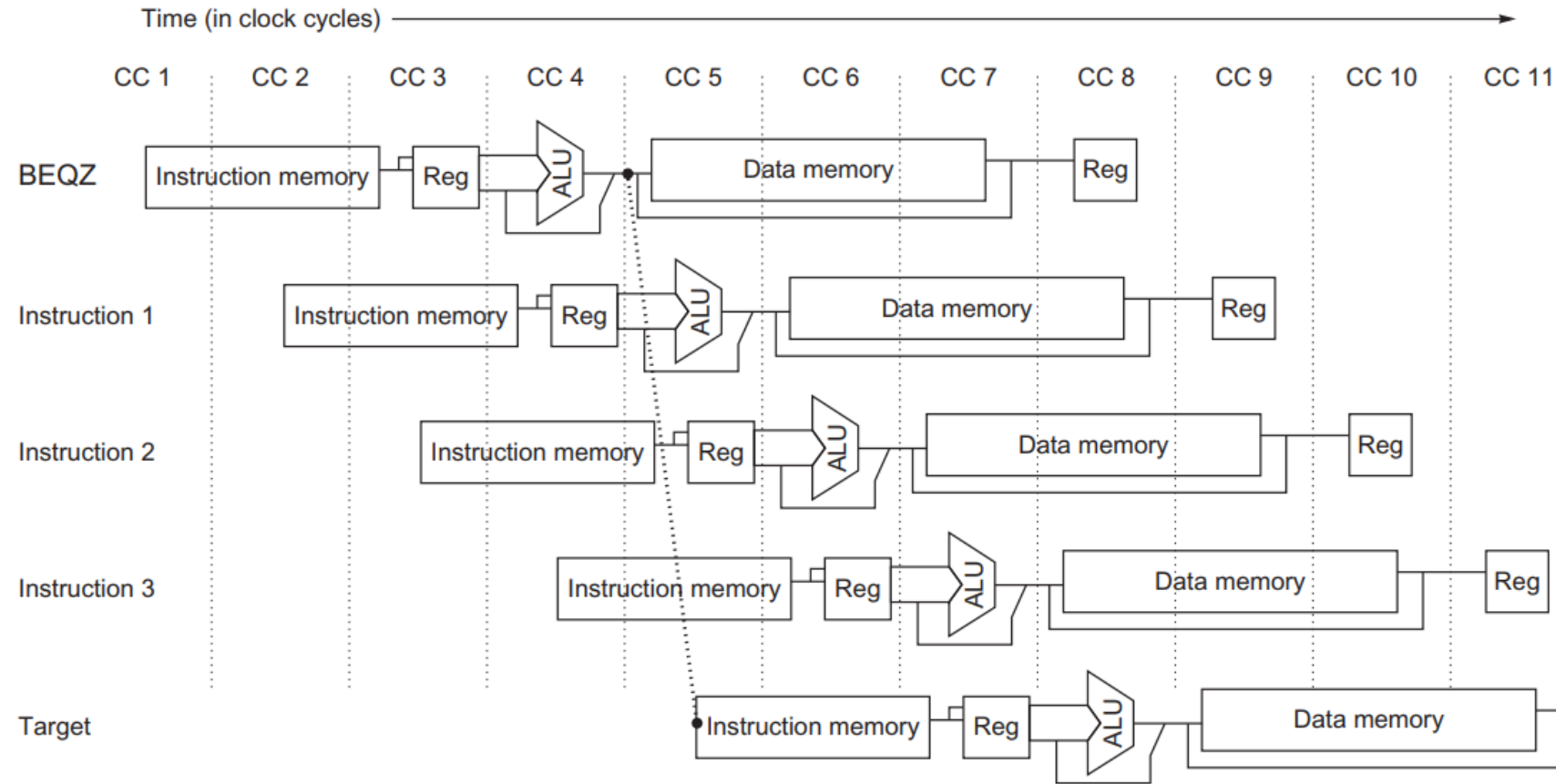


Figure C.39 The basic branch delay is three cycles, because the condition evaluation is performed during EX.

Pipeline interlocks enforce both the x1 branch stall penalty on a taken branch and any data hazard stall that arises from use of a load result. After the R4000, all implementations of MIPS processor made use of dynamic branch prediction.

Deeper Pipelines and Forwarding



In addition to the increase in stalls for loads and branches, the deeper pipeline increases the number of levels of forwarding for ALU operations. In our RISC V five-stage pipeline, forwarding between two register-register ALU instructions could happen from the ALU/MEM or the MEM/WB registers. In the R4000 pipeline, there are four possible sources for an ALU bypass: EX/DF, DF/DS, DS/TC, and TC/WB.

3 Cycle Branch Delay When Taken (x1 Interlock)

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Stall			Stall	Stall	Stall	Stall	Stall	Stall	Stall
Stall				Stall	Stall	Stall	Stall	Stall	Stall
Branch target					IF	IS	RF	EX	DF

Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Branch instruction + 2			IF	IS	RF	EX	DF	DS	TC
Branch instruction + 3				IF	IS	RF	EX	DF	DS

Figure C.40 A taken branch, shown in the top portion of the figure, has a one-cycle delay slot followed by a x1 stall, while an untaken branch, shown in the bottom portion, has simply a one-cycle delay slot. The branch instruction can be an ordinary delayed branch or a branch-likely, which cancels the effect of the instruction in the delay slot if the branch is untaken.

The Floating-Point Pipeline

The R4000 floating-point unit consists of three functional units: a floating-point divider, a floating-point multiplier, and a floating-point adder. The adder logic is used on the final step of a multiply or divide. Double-precision FP operations can take from 2 cycles (for a negate) up to 112 cycles (for a square root). In addition, the various units have different initiation rates. The FP functional unit can be thought of as having eight different stages, listed in [Figure C.41](#); these stages are combined in different orders to execute various FP operations.

There is a single copy of each of these stages, and various instructions may use a stage zero or more times and in different orders. [Figure C.42](#) shows the latency, initiation rate, and pipeline stages used by the most common double-precision FP operations.

FP Stages

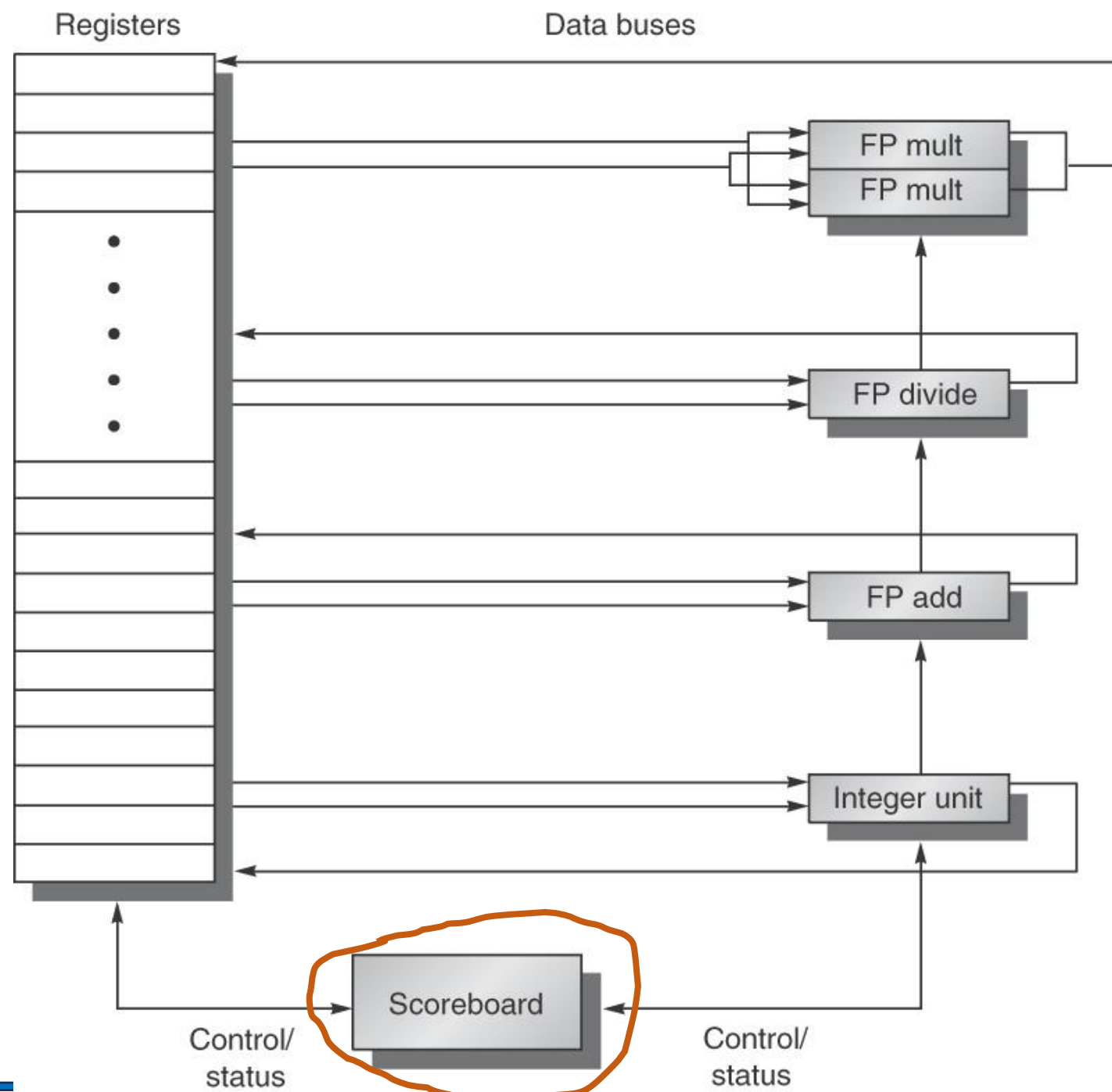
MIPS 4000

Stage	Functional unit	Description
A	FP adder	Mantissa add stage
D	FP divider	Divide pipeline stage
E	FP multiplier	Exception test stage
M	FP multiplier	First stage of multiplier
N	FP multiplier	Second stage of multiplier
R	FP adder	Rounding stage
S	FP adder	Operand shift stage
U		Unpack FP numbers

Figure C.41 The eight stages used in the R4000 floating-point pipelines.

FP instruction	Latency	Initiation interval	Pipe stages
Add, subtract	4	3	U, S + A, A + R, R + S
Multiply	8	4	U, E + M, M, M, M, N, N + A, R
Divide	36	35	U, A, R, D ²⁸ , D + A, D + R, D + A, D + R, A, R
Square root	112	111	U, E, (A + R) ¹⁰⁸ , A, R
Negate	2	1	U, S
Absolute value	2	1	U, S
FP compare	3	2	U, A, R

Figure C.42 The latencies and initiation intervals for the FP operations initiation intervals for the FP operations both depend on the FP unit stages that a given operation must use. The latency values assume that the destination instruction is an FP operation; the latencies are one cycle less when the destination is a store. The pipe stages are shown in the order in which they are used for any operation. The notation S + A indicates a clock cycle in which both the S and A stages are used. The notation D²⁸ indicates that the D stage is used 28 times in a row.



Computer Architecture

CS 622 / EE 520

Spring 2026

Lecture 9
Shahid Masud

- Quick Review of Pipeline and Hazards
- Scoreboard in Dynamic Branch Prediction
- Instruction Level Parallelism – Chapter 3
- Parallelism through Loop Unrolling
- Advanced Branch Predictors

Quiz 2
Next Lecture

Reading Notes

Readings:
Appendix C of course textbook
Chapter 3 of course textbook

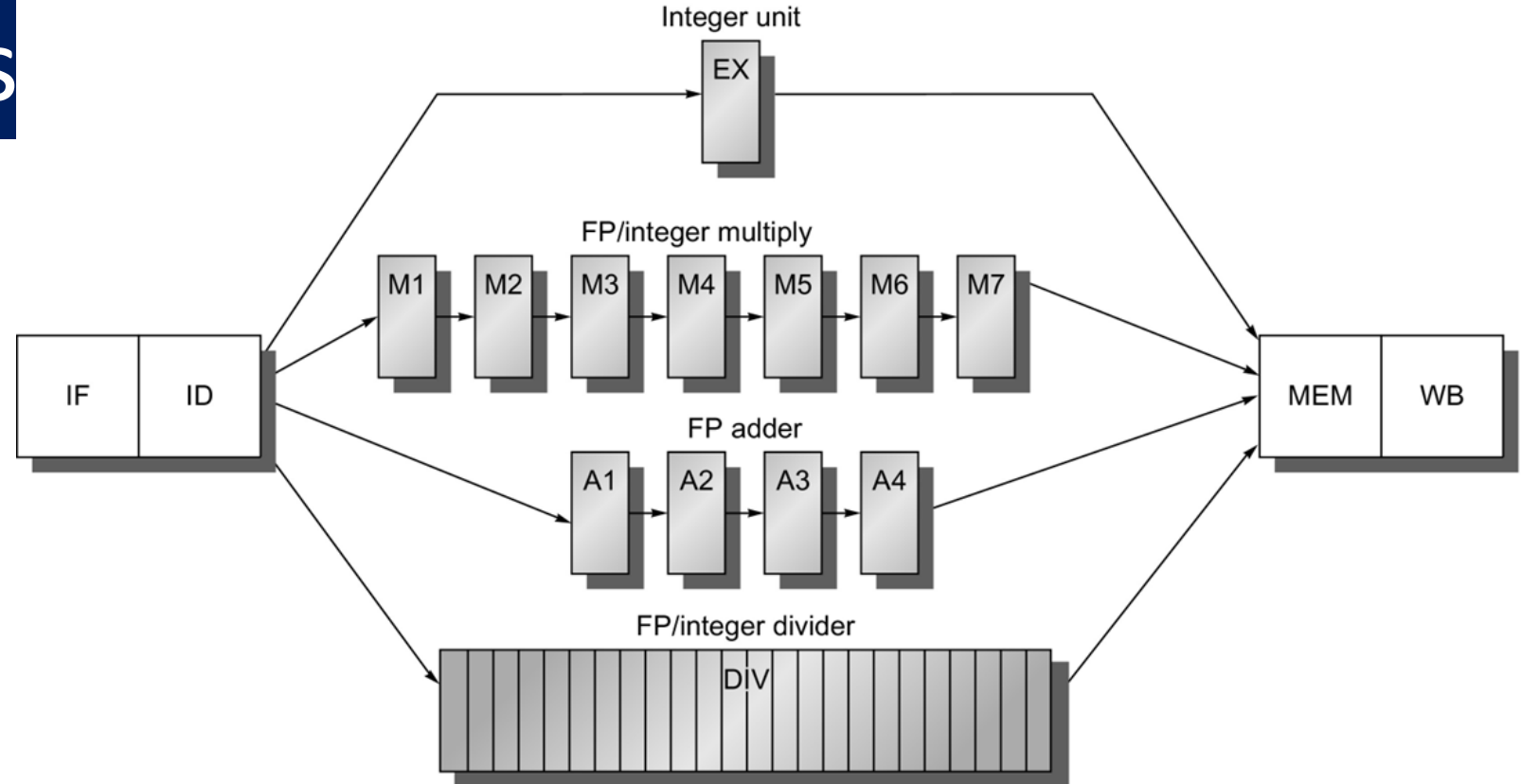
Pipelining topics covered so far



- RISC ISA
- 5 stage Integer Pipelined CPU
 - Performance
 - Hazards
- Stalls to improve RAW, WAR, WAW Data Hazards
- Reordering and NoOp in Software / Compilers
- Forwarding (or Bypassing) to Improve Data Hazards
- Interlock feature in Pipelined Hardware Architecture
- Control / Branch Hazards
 - Static / Dynamic / Flush / Restart
 - Branch Prediction
- Dealing with Exceptions / Interrupts in Pipelined Architecture
 - Status and Mask Registers
- Multicycle Floating point Pipelined RISC Architecture
 - Introduce Out of Order Execution
 - Introduce Scheduling of Instructions
- Case study of MIPS R4000 Pipeline
 - Instruction Memory and Data Memory with Cache support

Adding Registers

Multicycle Pipeline



The structure of the pipeline in Figure C.30 requires the introduction of the additional pipeline registers (e.g., A1/A2, A2/A3, A3/A4) and the modification of the connections to those registers. The ID/EX register must be expanded to connect ID to EX, DIV, M1, and A1; we can refer to the portion of the register associated with one of the next stages with the notation ID/EX, ID/DIV, ID/M1, or ID/A1. The pipeline register between ID and all the other stages may be thought of as logically separate registers and may, in fact, be implemented as separate registers. Because only one operation can be in a pipe stage at a time, the control information can be associated with the register at the head of the stage.

Hazards and Forwarding in Longer Latency Pipelines

There are a number of different aspects to the hazard detection and forwarding for a pipeline like that shown in Figure C.30.

1. Because the divide unit is not fully pipelined, structural hazards can occur. These will need to be detected and issuing instructions will need to be stalled.
2. Because the instructions have varying running times, the number of register writes required in a cycle can be larger than 1.
3. Write after write (WAW) hazards are possible, because instructions no longer reach WB in order. Note that write after read (WAR) hazards are not possible, because the register reads always occur in ID.
4. Instructions can complete in a different order than they were issued, causing problems with exceptions; we deal with this in the next subsection.
5. Because of longer latency of operations, stalls for RAW hazards will be more frequent.

8 Stage Pipeline of MIPS R4000

- IF: First half of instruction fetch; PC selection actually happens here, together with initiation of instruction cache access.
- IS: Second half of instruction fetch, complete instruction cache access.
- RF: Instruction decode and register fetch, hazard checking, and instruction cache hit detection.
- EX: Execution, which includes effective address calculation, ALU operation, and branch-target computation and condition evaluation.
- DF: Data fetch, first half of data cache access.
- DS: Second half of data fetch, completion of data cache access.
- TC: Tag check, to determine whether the data cache access hit.
- WB: Write-back for loads and register-register operations.

SUPERPIPELINING

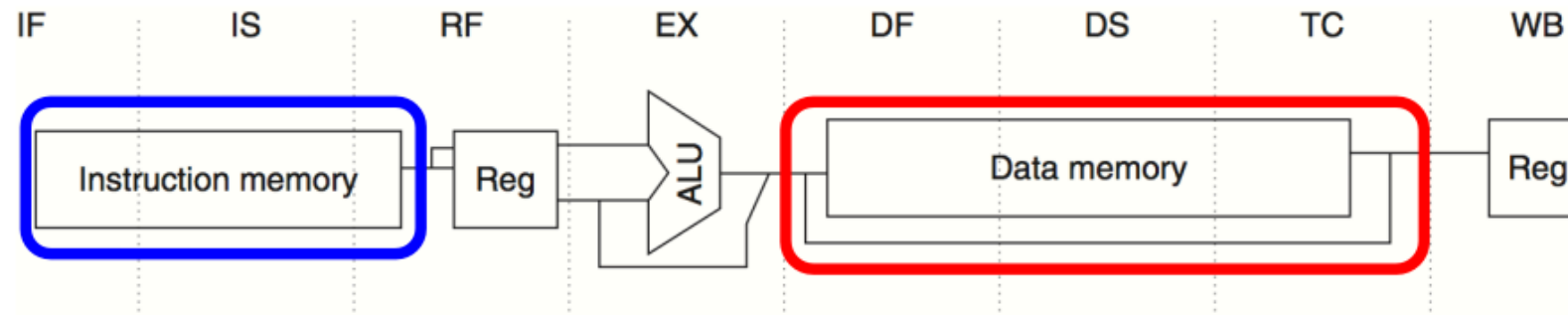


Figure C.36 The eight-stage pipeline structure of the R4000 uses pipelined instruction and data caches. The pipe stages are labeled and their detailed function is described in the text. The vertical dashed lines represent the stage boundaries as well as the location of pipeline latches. The instruction is actually available at the end of IS, but the tag check is done in RF, while the registers are fetched. Thus, we show the instruction memory as operating through RF. The TC stage is needed for data memory access, because we cannot write the data into the register until we know whether the cache access was a hit or not.

Figure C.39 shows that the basic branch delay is three cycles, because the branch condition is computed during EX. The MIPS architecture has a single-cycle delayed branch. The R4000 uses a predicted-not-taken strategy for the remaining two cycles of the branch delay. As Figure C.40 shows, untaken branches are simply one-cycle delayed branches, while taken branches have a one-cycle delay slot followed by two idle cycles. The instruction set provides a branch-likely instruction, which we described earlier and which helps in filling the branch delay slot.

Basic Branch Delay

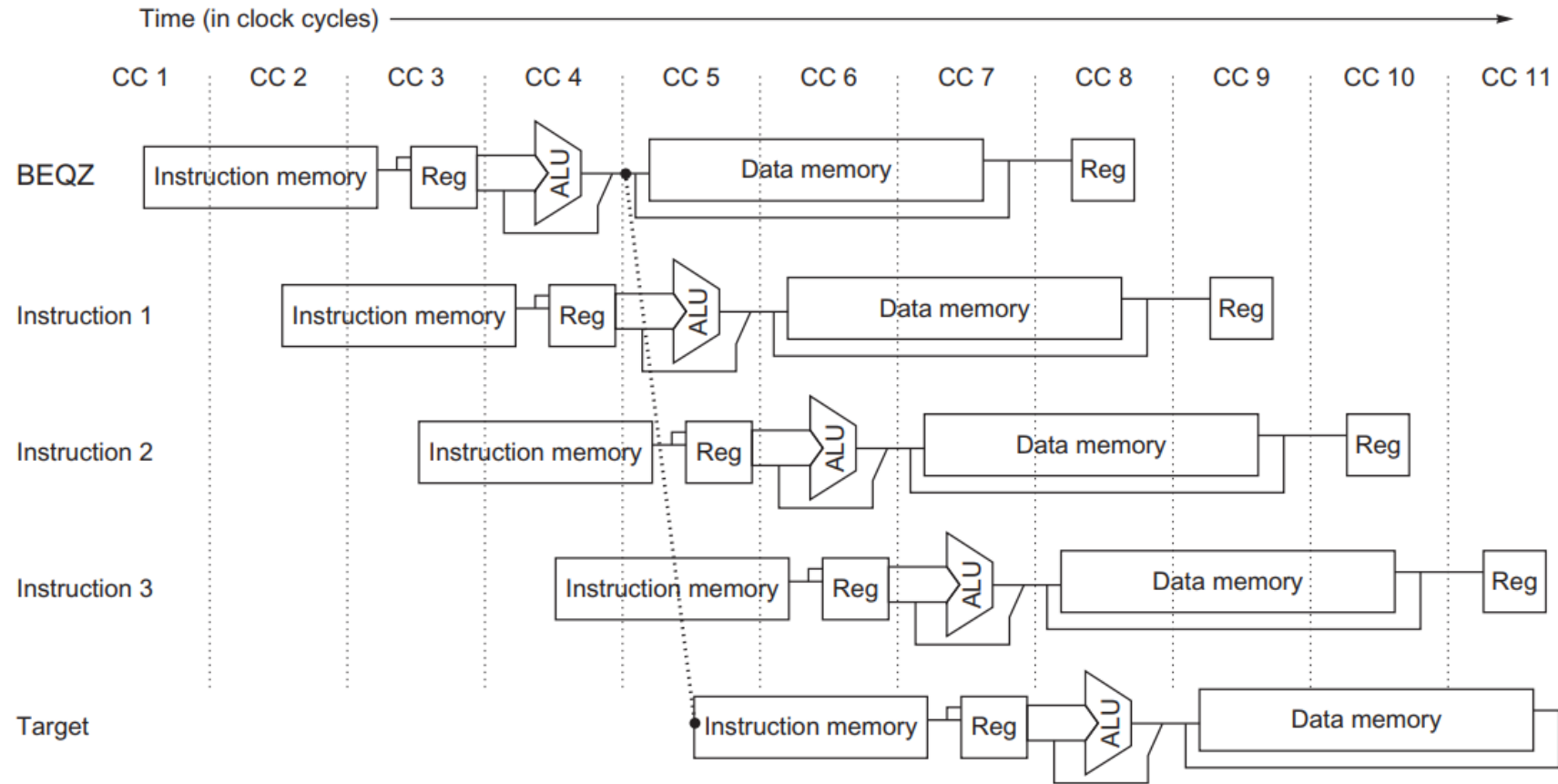


Figure C.39 The basic branch delay is three cycles, because the condition evaluation is performed during EX.

Pipeline interlocks enforce both the x1 branch stall penalty on a taken branch and any data hazard stall that arises from use of a load result. After the R4000, all implementations of MIPS processor made use of dynamic branch prediction.

Deeper Pipelines and Forwarding



In addition to the increase in stalls for loads and branches, the deeper pipeline increases the number of levels of forwarding for ALU operations. In our RISC V five-stage pipeline, forwarding between two register-register ALU instructions could happen from the ALU/MEM or the MEM/WB registers. In the R4000 pipeline, there are four possible sources for an ALU bypass: EX/DF, DF/DS, DS/TC, and TC/WB.

Forwarding Example MIPS 4000

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
ld x1,...	IF	IS	RF	EX	DF	DS	TC	WB	
add x2,x1,...		IF	IS	RF	Stall	Stall	EX	DF	DS
sub x3,x1,...			IF	IS	Stall	Stall	RF	EX	DF
or x4,x1,...				IF	Stall	Stall	IS	RF	EX

Figure C.38 A load instruction followed by an immediate use results in a x1 stall. Normal forwarding paths can be used after two cycles, so the `add` and `sub` get the value by forwarding after the stall. The `or` instruction gets the value from the register file. Because the two instructions after the load could be independent and hence not stall, the bypass can be to instructions that are three or four cycles after the load.

3 Cycle Branch Delay When Taken (x1 Interlock)



Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Stall			Stall	Stall	Stall	Stall	Stall	Stall	Stall
Stall				Stall	Stall	Stall	Stall	Stall	Stall
Branch target					IF	IS	RF	EX	DF

Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Branch instruction + 2			IF	IS	RF	EX	DF	DS	TC
Branch instruction + 3				IF	IS	RF	EX	DF	DS

Figure C.40 A taken branch, shown in the top portion of the figure, has a one-cycle delay slot followed by a x1 stall, while an untaken branch, shown in the bottom portion, has simply a one-cycle delay slot. The branch instruction can be an ordinary delayed branch or a branch-likely, which cancels the effect of the instruction in the delay slot if the branch is untaken.